

固定调度机制下被抢占请求的原生恢复方式选择

阶段报告：正常容量下的恢复覆盖与零动作时延漂移

独立研究会话 E

2026 年 10 月 7 日

摘要

在支持 Host KV 卸载的推理服务中，被抢占请求再次获得恢复机会时，可以加载已有合法 KV，或利用原生重算重建状态。本文固定恢复目标、victim 规则、队列、服务量子数和准入，仅研究这一次执行方式的选择及其完整服务代价。当前完成了 vLLM 原生路径的薄适配，以及正常容量下 64、192、256 请求三档共八格、1536 次请求执行。各档跨臂输出逐 token 一致，但正式测量均无抢占或恢复动作；实际 GPU KV 容量为 72.1015625 GiB，三档峰值占用分别为 20.44%、43.05%、48.44%。192 请求组在相同调度工作下出现 31.8% 的完工时间漂移，256 请求 ABBA 中首末同策略运行相差 19.0%。这些差异不能归因于恢复选择，也不构成自适应方法收益。本文区分正式测量与固定输出预热，保留全部已支付 STORE，并报告尚未激活的合法恢复域。唯一后继自然长摘要负载已冻结但未运行；恢复分支正确性、状态信号增量信息和策略收益仍待真实事件验证。

1 问题与可检验范围

设请求 r 在时刻 t 已收到的输入为 P_r ，已经对外产生的 token 为 $Y_r^{\leq t}$ 。恢复必须保留的已知历史为

$$H_r(t) = P_r \| Y_r^{\leq t}, \quad n_r(t) = |H_r(t)|. \quad (1)$$

选择器只读取当前和历史观测，不能使用未来输出长度，也不能先执行两条路径再选择较快结果。本文考虑如下两个动作：

- $a = \text{Host}$ ：加载 Host 中已经就绪、与当前历史一致的原生 KV 前缀，并由原生路径补齐未命中的必要尾部。
- $a = \text{Recompute}$ ：保留同一个请求及其已知 token 历史，经原生调度器和模型执行器重建必要 KV，随后继续生成。

Host 命中并不要求覆盖全部历史；两条路径最终都必须到达原请求下一次合法生成所需的完整状态。研究对象不是重新提交一个新请求，也不是把已经生成的文本当作新的回答重新采样。

第一版固定恢复目标、抢占对象选择、FCFS 队列、token 预算、准入检查和传输实现。动作导致后续系统状态自然变化是实验结果的一部分；不能为了强行对齐两臂的执行轨迹，再修改调度器。对照固定的是外部到达序列、资源预算和机制参数，而不是要求两次运行具有完全相同的抢占事件集合。

待检验假设是：孤立恢复速度可能没有反映传输排队、GPU 计算竞争及其他请求的停顿。不过，这只是实验动机；本文不预设存在交叉点，不预设状态感知规则优于最佳固定动作。若当前运行域始终由一个动作占优，或只有一条路径可合法执行，应直接报告该边界。

2 成本口径与正确性边界

令 t_r^{next} 为一次已提交恢复选择之后该请求首次新输出的时间。局部观测为

$$L_r^{\text{next}} = t_r^{\text{next}} - t_r^{\text{decision}}. \quad (2)$$

该时间包含真实排队、传输、重算和调度等待，不能用孤立复制时间替代。若请求在下一输出前再次被抢占，多次选择可能对应同一次输出；分析器保留这种关联，不将这些记录当作独立因果样本。

全局观测使用所有请求的外部到达 A_i 、完成 C_i 以及连续输出时间 $O_{i,k}$ ，至少报告平均完成时延、整体完工时间以及生成间隔：

$$\bar{F} = \frac{1}{N} \sum_{i=1}^N (C_i - A_i), \quad M = \max_i C_i - \min_i A_i, \quad G_{i,k} = O_{i,k} - O_{i,k-1} \ (k > 1). \quad (3)$$

同时报告 TTFT、每请求最大停顿、间隔分位数和未完成请求。正常容量主组保留自然 EOS，预先声明的输出上限只限定合法生成范围；必须同时报告实际输出长度、EOS/长度截断数量和输出差异，不能假设各臂工作量恒等。固定输出只用于另行标记的等工作量诊断。自然停止本身也不构成任务质量或生产 goodput 证据。

已经发生的 STORE 是沉没成本。 在 t 之前，为请求保存 KV 的计算与传输已经真实发生。无论随后选择哪个动作，该历史成本都保留在整段服务统计中；比较当前动作的增量影响时，不能将其再次计入 Host 分支，也不能将其记作重算节省。两臂均保留同一原生 STORE 机制。动作以后新增的计算、LOAD、STORE 和排队仍由真实执行统计。异步传输与计算可能重叠，CUDA 事件时间可用于诊断，却不能简单相加来重建端到端墙钟时间。

完整状态容量。 设每个 GPU KV block 容纳 b 个 token，当前已知历史至少需要

$$B_r = \lceil n_r(t)/b \rceil. \quad (4)$$

当前薄适配器仅在 $B_r + B_{\text{reserved}} + B_{\text{watermark}} \leq B_{\text{free}}$ 时允许改变原生结果，并继续执行原生分配器的全部检查。运行器打开完整序列容量约束。该条件不是声称只加载较短 Host 前缀便可减少完整状态所需容量，也不替代模型执行的其他内存要求。

请求连续性。 选择前后核对对象身份、全部已知 token、采样参数对象、输出进度、状态及停止原因，记录前缀哈希。输出必须单调扩展已有序列，重建未完成时不得冒出新的输出 token。首轮使用 greedy 路径；不据此宣称已经验证随机采样的 RNG 状态等价。已完成三档的正式测量均未执行恢复动作，因此其输出一致性不能代替恢复状态不变量的实测验证。即使历史完全相同，批形状改变也可能引入数值差异，故跨臂 token 差异应如实保留，并在后续自然任务中单独评估质量。

3 原生接口与薄适配

实现依据本地保存的 vLLM 0.26.0 源码快照。原生 connector scheduler 的 `get_num_new_matched_tokens` 返回可用外部前缀；随后 `update_state_after_alloc` 在 GPU 分配后准备加载，并调用 `manager.prepare_load`。原生主调度器的 `_preempt_request` 重置已计算进度，请求对象仍持有 `all_token_ids`，供调度器恢复计算。源文件路径和哈希由项目的 `native_sources/` 与 `source_manifest.json` 保存。

`selector.py` 在原生 lookup 之后作一次小范围适配。只有已经抢占过、存在正的就绪 Host 命中且满足共同容量条件的请求进入可选集合。Host 策略原样保留命中；Recompute 策略将 connector 的命中元数据同步归零并返回 `(0, False)`，让同一个原生请求进入正常计算路径。选择器不分配另一套 KV，不实现新的重算执行器，也不在选择前执行传输或模型计算。双方都经过原生 lookup，保留其共同的查找行为。

lookup 可能被尝试多次而未真正获得资源，因此尝试次数不等于执行决策数。当前记录以 `update_state_after_alloc` 为提交边界，分别保存偏好、返回结果与实际外部 token 数。尚在等待本请求原生传输时沿用原生延期；Host 未命中时沿用原生重算；共同容量不足时保留原生结果与准入检查。这些 fallback 独立计数，不作为选择器成功样本隐藏。

适配依赖私有字段，故版本变化可能破坏假设。运行时限定原生 CPU offloading、单 KV group、非滑窗及关闭 APC，并检查预热后缓存重置、实际 GPU/Host 存储字节数和 Host pinned 属性。断言失败应留下失败记录并停止该实验格。当前可运行策略为固定 Host、固定 Recompute 及显式长度阈值；尚未加入动态规则。已完成八格正式服务的策略动作全部为零，尚无由选择器提交的合法双分支对照，不能称为两种恢复方式均通过

GPU 正确性验证。256 请求的固定输出预热虽出现原生 LOAD，但选择器关闭且不记录恢复事件，不能用它补足正式分支验证。

4 首轮实验协议

表 1 给出已完成三档正常容量实验的统一协议，结果见第 7 节。GPU KV 采用 `gpu_memory_utilization=0.9` 下的原生 profiling，不设置显式 KV 字节上限；记录实际可用 block 数和张量存储字节数。每组只初始化并 profile 一个引擎，各臂复用该引擎；组间重新初始化并分别核对实际容量，不以相同 utilization 参数代替容量证据。本文主结果不包含显式限容实验。

表 1: 正常容量首轮配置；最终以每组日志及资源断言为准。

项目	配置
设备与模型	RTX PRO 6000 Blackwell Server Edition；服务器已有 OLMoE 本地快照
软件与执行	vLLM 0.26.0；PyTorch 2.11.0；Transformers 5.17.0；BF16、eager、原生 Triton MoE
KV 预算	GPU 原生 0.9 profiling；pinned Host 32 GiB；记录 chunk 舍入后实际容量；APC、推测解码关闭
调度与准入	FCFS；最多 256 请求；每步 2048 token；完整当前历史必须可容纳
工作负载	64、192、256 请求三档；固定 1 短:3 长；相同档位各臂输入和外部到达相同，外部到达均为 0
输入与输出上限	短 GSM8K 输入 661–约 800 token、输出上限 256；长完整 QA 输入 2699–3271 token、输出上限 512
生成与顺序	greedy、自然 EOS，256/512 为上限；低负载和接近容量候选各 Host/Recompute 两臂，高压候选用 Host/Recompute/Recompute/Host
引擎与隔离	同组各臂复用同一引擎；每臂前相同完整预热和缓存重置；组间释放 GPU

三组使用同一已核验 GPU，设备 UUID、软件版本与运行时源文件哈希随原始结果保存；汇总器的配置指纹本身不替代源码哈希核验。

工作负载由 `inputs_pro6000/freeze_receipt.json` 在 CPU 侧冻结。短提示来自公开 GSM8K test 的 64 个选定样本及既有八示例前缀 [8]；长提示来自 LongBench `multifieldqa_en` 的 17 条完整自然 QA [9]，包含原上下文和问题，不通过截断或重复拼接文本构成长输入。长任务循环复用这些样本，但每次提交是独立请求并使用独立缓存身份；关闭 APC，不能借重复提示获得 GPU 前缀复用。三档分别含 33、65、81 个不同提示，是重复自然请求的压力探针，不能作为独立任务泛化或质量结论。

表 2: 冻结的三档负载。容量列仅是假设所有请求同时驻留且均生成到上限时的完整 KV 上界估计，非实测分配或抢占证据。

候选档位	请求数	短/长请求	不同提示数	上界估计 (GiB)
低负载	64	16/48	33	22.35
接近容量	192	48/144	65	67.23
高压	256	64/192	81	89.66

表 2 的估计按 16-token block 舍入，来自冻结清单；实际驻留时间、原生准入与完成顺序可能使峰值占用显著低于此值。“接近容量”和“高压”是候选负载标签，须由实际 block 使用、队列、抢占与合法 Host 命中记录确认。Host 预算也不保证恢复命中，原生驻留和淘汰决定可用前缀。

同组每臂开始前，先排空上一 episode 的原生传输并重置 Host/GPU 缓存；再关闭选择器，用该档全部请求执行相同原生 Host 预热，排空 STORE/LOAD 后再次重置。当前共同预热仍忽略 EOS 并生成到各自上限，用于覆盖并发批形状；这些输出不属于正式测量。复位后正式主组使用自然 EOS，不能把预热期间的抢占当作主组恢复。检查请求队列、connector 作业与缓存空闲基线，防止前臂状态泄漏。计时从重置后的正式 episode

开始；版本、配置、输入清单、预热成本、资源、状态和原始事件均单独落盘。若预热成本需要缩减，应依据运行证据另行冻结相同的新预热方案，不在组内改动。不能事后删除不利运行；超时或断言失败时保留部分轨迹与失败原因。

每组持有共同 GPU 锁，初始化及各臂测量前后核对设备与计算进程，结束后释放 GPU。锁等待和初始化不计入正式 episode；所有中止另行记录，不作为已完成测量格。

`run_cell.py` 记录外部到达与实际入队时间、每次原生调度的计算区间、抢占、选择/提交、每个新输出及完成时刻，以及原生 LOAD/STORE 报告。输出时刻取本地同步 `engine.step()` 返回后的观测，包含 Python 驱动与插桩开销，尚不是网络流式客户端时延。`analyze.py` 汇总上述局部与全局指标，并区分 lookup 尝试、提交和 fallback。所有统计以真实服务为基础，不离线扣除估计开销构造另一个动作。

低负载首先检验正常容量基线与追踪完整性；高压 ABBA 再检查方向的反序稳定性。正常容量下没有抢占或没有两路径均合法的恢复事件也是有效结果，恢复时延应记为不适用，不能填零后宣传收益。一次突发和重复自然提示不足以支持稳定尾分位数或统计显著性论断。两臂的事件数可能不同；不得将跨运行、同长度但不同系统历史的恢复事件直接视为随机配对。可对照的是外部请求及整段服务，事件分组只作为解释性诊断。

5 最小在线规则与后续推进条件

若真实对照出现可重复且值得利用的差异，先增加长度/成本阈值

$$\pi_\ell(n) = \begin{cases} \text{Recompute}, & n \leq \tau, \\ \text{Host}, & n > \tau, \end{cases} \quad (5)$$

并用独立校准运行确定 τ ，随后固定到验证运行。若测量不支持这一方向，应调整或弃用该假设，而不是强制制造阈值交叉。硬件成本基线只比较未来的 LOAD/必要尾部计算与重建成本。

当前轨迹同时留有待处理 LOAD 数、队列规模和近期原生步骤时间，目的是诊断，不代表在线规则应同时使用所有特征。由于正式恢复样本为零，目前不拟合状态模型。只有获得可重复的合法动作差异后，才选择一个有证据的信号 z_t ，例如已暴露的传输等待或近期步骤耗时；分支方向及阈值必须由先前数据确定，在验证序列到达前冻结。禁止使用未来输出长度，禁止将事后最优动作算作在线收益。

比较对象必须包含最佳固定方式、固定长度/成本阈值和增加一个状态信号的规则。若规则退化为固定阈值，应直接报告等价性；若状态在控制长度后没有增量解释力，应删除该状态模型。若局部恢复变快而全局完成或生成间隔恶化，下一步先检查该动作占用了哪些请求的执行机会，而不是继续扩大参数搜索。

信号稳定后再扩展到独立任务、不同到达形状与至少一个独立负载；报告任务质量、完整开销及所有失败。需要另行测量有无插桩的差异、选择器运行开销与随机采样连续性。主组 GSM8K 与长 QA 虽保留自然 EOS，17 条长 QA 的循环独立提交仍不增加不同任务数量，不能直接提供泛化或问答质量结论。--fixed-output 仅用于明确标记的等工作量诊断，不能用其强制生成的压力替代自然主组证据。

6 相关工作与研究定位

CacheOPT 按模型和硬件对长度建立重算、swap 成本函数，其比较包含 swap-out 和 swap-in；系统还联合改变其他 KV 分配和调度机制 [1]。Stream2LLM 在 preemption 时采用硬件成本选择，其实现明确使用闲置系统静态 profile，并独立比较调度与驱逐策略 [2]。这两者是成本基线来源；在本文 STORE 已支付的边界上必须重新标定。本次补充核对了 Stream2LLM 官方 commit 5e2763d 的 v1 调度器与 engine core：前者在抢占时比较两个静态成本函数，后者对重算均值和 swap-in 加 swap-out 的均值分别拟合二次曲线。其函数自变量为当前分配调用的 `num_new_tokens`，不能未经核对就改述为 victim 前缀长度。本文尚未复现其整套系统；将其成本思想移至 STORE 已付的恢复边界，需要单独声明并重新校准。

UnboxKV 已实测并发、GPU KV 容量与模型影响 swap/recompute 排名，且指出重复 prefill 会影响调度 [3]。因此，观察到负载相关排名变化本身不是本文可主张的新发现。Cake 双向并行计算前缀与加载后缀，并

调整与其他请求共享算力的优先级 [4]; CacheGen 根据网络条件在不同压缩 KV 与文本重算之间适配 [5]。这些工作已经建立资源状态与计算/加载选择之间的关系, 其混合执行、编码或调度变化超出本文第一版动作空间。

LUMEN 在故障恢复时根据 worker 负载, 必要时放弃 checkpoint 本地性并改派请求重算 [6]。其目的地选择与本文固定目的地的动作不同, 但使“首次负载感知恢复”这一表述不成立。ShadowServe 研究 KV 解压与模型计算的干扰, 说明局部数据路径指标不足以替代完整服务指标 [7]; 本文没有解压步骤, 不能沿用其干扰量级。本文当前定位是一个受控问题与最小实现, 是否具有超出强近邻的实证价值仍取决于后续真实数据。

7 正常容量结果、证据状态与唯一后继

八格真实服务完成。 execution/oct07_low_r02/、oct07_near_r02/ 和 oct07_high_r01/ 分别保存 64、192、256 请求组。前两组各两臂, 后一组为 ABBA, 共八格、1536 次正式请求执行。各格均完成, 无分析错误、失效或未完成格; 输入、固定配置、实际资源及预热重置按组核对。独立原始轨迹复算保存在 low64_results.txt、near192_results.txt 和 high256_results.txt。这些是请求执行次数, 不是 1536 个独立任务或统计重复。

表 3: 全部正式测量。H/R 仅表示所配置的 Host/Recompute 策略, 数字表示组内顺序; 八格实际恢复动作全部为零, 表中差异不是恢复收益。

请求数	臂	输出 tokens	完工 (s)	均值 (s)	P95 (s)	P99 间隔 (ms)
64	H0	8,694	9.693	3.957	7.541	35.726
64	R1	8,694	9.583	3.979	7.567	35.480
192	H0	28,648	19.548	9.966	17.783	48.592
192	R1	28,648	25.769	11.160	23.529	58.165
256	H0	37,730	27.289	12.298	22.757	59.638
256	R1	37,730	22.945	11.478	20.948	51.881
256	R2	37,730	22.872	11.455	20.906	51.856
256	H3	37,730	22.927	11.499	20.970	52.474

表 3 的均值及 P95 均为从固定外部到达计算的完成时延, 生成间隔不含 TTFT。各档的逐请求 greedy 输出序列跨臂全同, 完成/停止元数据也相同; 256 档的六个两两比较均为 256/256 一致。每臂以长度上限结束的请求数在三档分别为 3、14、20, 其余自然停止。输出 cap 之和分别为 28,672、86,016、114,688, 显著大于实际输出, 不能以 cap 代替工作量。该一致性检查既不是任务质量评分, 也不能验证尚未执行的恢复状态不变量。

合法恢复域仍为空。 三组独立初始化所得实际 GPU KV 均为 72.1015625 GiB、36,916 blocks, pinned Host 均为 32 GiB、16,384 blocks。八格的抢占请求数、恢复尝试、Host-ready 事件、共同可行事件、已提交动作和 fallback 均为零, LOAD 字节与传输数也均为零。因此 decision-to-next-output 没有样本, 应记为不适用, 而不是零时延。表 4 给出实际观测; “接近容量”和“高压”候选标签均未在正式自然输出下形成恢复竞争。

表 4: 各档正式测量的实际峰值及每臂 STORE; 同档各臂相同。GPU 峰值取 schedule/step/preempt 边界, 含初始常占的 1 block; Host 峰值取 step 边界。

请求数	GPU 使用 blocks	GPU 占比	Host blocks	STORE (GiB)
64	7,545	20.4383%	10,137	19.798828
192	15,892	43.0491%	16,384	59.876953
256	17,881	48.4370%	16,384	79.804688

这些占用不是完整 allocator 高水位或物理 GPU 总显存占比。Host 常驻满额也不意味着发生恢复: 192/256 档持续 STORE 与原生淘汰可以发生在零 LOAD 的服务过程中。最早进入的自然任务可在后续 prefill 建立 KV 前完成, 因而增加外部请求数不必然增加同时驻留到上限的状态; 这是一种与结果相容的解释, 尚不能单独确认为全部容量变化的原因。

完整保留 STORE。 三档每臂 STORE 字节数分别为 21,258,829,824、64,292,388,864、85,689,630,720，已报告传输数分别为 685、2214、2924。按表 3 的臂顺序，CUDA-event 持续时间和分别为 64 档 0.410424/0.413987 s，192 档 1.264007/1.302383 s，256 档 1.692707/1.673955/1.678274/1.685173 s。统计保留整个正式 episode 及 drain，不含共同预热与初始化，未从重算配置扣除已支付 STORE。192/256 档不仅总字节相同，每批 STORE 的字节和尺寸序列也相同。事件时间可以重叠，不是关键路径等待；不能用其与墙钟作简单加减来排除传输排队的影响。包含空 LOAD 字段的元数据报告数不等于 LOAD 次数。

自然退休与实际驻留。 在三档首次达到观测 KV 峰值时，已有 14、94、145 个请求完成，全部为自然停止；已开始但未完成的请求仅为 50、98、109 个。256 请求组的峰值可分解为 145 个已完成、106 个已输出未完成、3 个正在 prefill 及 2 个尚未开始的请求。长 QA 的平均实际输出仅为 153.1、165.1、162.9 token。轨迹支持自然退休限制同时驻留的解释，但不证明容量已经平台化：192 到 256 时峰值仍增加 5.39 个百分点。新摘要任务只检验更长自然输出是否改变该边界，不保证出现恢复。详细计数见 `capacity_retirement_note.txt`。

同工作量下的零动作漂移。 按外部请求身份规范化内部随机 ID 后，192 档全部 728 步、256 档全部 784 步在各自组内完全相同：包括 scheduled 请求顺序、计算位置区间、token 数、已知/已生成进度和 schedule 后空闲 block。两档原生调度 token 数分别为 491,770 和 655,437；运行/等待队列大小、GPU 空闲量、待处理 LOAD 数和 Host 常驻量也逐步一致。于是观察到的时间差不能解释为选择器改变了这些工作。

192 档 R1 比 H0 多 6.220755 s (31.8%)，其中 `engine.step()` 时间和多 6.237251 s；step 350–599 占主要差异约 4.234116 s。R1 的 step 612 为 34.348 ms，下一步降到 13.341 ms，但两步仍同为 9 请求、9 个 scheduled token，完成请求数也没有改变。256 档首末同为 Host 的 H0/H3 相差 4.361749 s (19.0%)；最后 184 步贡献 step 总差的 60.2%，其余三臂完工时间仅相差 0.072711 s。两组漂移跟随不同运行位置，既不能称为固定恢复方式排名，也不能通过 ABBA 按策略平均转化为收益。

约 89% (192 档) 和 90% (256 档首末 Host) 的 step 时间差位于 schedule 后的 trace 时间戳与 step 返回之间。该区间仍混有主机派发、GPU 执行/等待和输出处理，不能称为已测 GPU 计算变慢。GPU 端点记录只见允许进程，cgroup 累计限额/OOM 事件为零；它们不能证明整个测量期间没有 CPU 争用、时钟变化或等待。现有证据定位了持续慢步，却不足以确定根因。

预热与正式恢复必须分开。 256 档四次共同预热各生成固定 114,688 tokens、执行 1057 步，GPU 最小空闲 block 为零，日志确有原生 LOAD；正式测量仅生成 37,730 tokens，峰值占用 48.44%，且无恢复。预热期间 `selector.enabled=False`，空抢占列表表示没有记录，不能推断预热零抢占，也不能将其 LOAD 补算成正式选择器覆盖。四次预热完成时间为 56.839/56.326/57.711/61.539 s，规范化调度相同；最后一次预热较慢而其正式测量较快，不能用简单的“后运行更热”解释全部漂移。192/256 组可见的 JIT warning 均在首臂预热，未发现正式测量中的数秒单步尖峰；这不排除未记录开销，但不支持将持续漂移直接归于该次编译。

证据状态。 已验证的是：正常容量三档的服务链、原始追踪、共同资源/预热复位和零恢复下的输出一致性，以及 192/256 档逐步调度的一致性；已观测的是：这些自然负载没有激活恢复域，以及不可归于策略的同工作量时延漂移。尚未验证的是：选择器两条分支的合法事件执行与恢复正确性、恢复时延、固定方式排名、长度阈值效果、状态信号增量信息、选择器/插桩开销、随机采样连续性 & 任务质量。因此本稿是问题与证据边界的阶段报告，不是完成了收益验证的自适应方法论文。

唯一后继：冻结的自然长摘要容量探针，尚未运行。 `natural_pressure_next.txt` 与 `inputs_natural_summary256/freeze_receipt.json` 固定了下一组 256 请求：保留 64 条原 GSM8K 短请求，将 192 条长请求改为三篇已保存完整文档的详细摘要，各篇独立提交 64 次。指令要求忠实、600–750 词、6–8 个小节；全文不截断、拼接或重复填充。三个完整 prompt 分别为 3052、2871、2736 tokens，长输出 cap 为 1024，与 4096 上下文上限相容。仍使用自然 EOS (`min_tokens=0`、`ignore_eos=False`)，不能保证实际输出长度；三篇文档的重复提交不是独立任务泛化证据。

该输入已在 CPU 冻结, 本文快照尚无 GPU 执行结果, 不计入前述八格。组内保持正常显存、相同机制和 Host/Recompute/Recompute/Host 顺序; 唯一问题是自然长输出能否实际延长 KV 驻留并产生合法双分支恢复。若仍无恢复, 保留空适用域结果, 不降低显存或强制生成来把该自然组改为阳性。输出触顶、摘要事实覆盖与截断须独立检查, 较少退休或更长文本不自动代表有效任务输出。

鉴于同工作量漂移已在两个正式组复现, 已提交的下一组统一加入每步 `process_time/thread_time` 增量, 以及独立 1 Hz GPU 时钟、功率、温度、利用率和进程采样, 并记录正式 episode 的 `monotonic/UTC` 起点对应关系。此改动只用于旁观测, 不调整策略或同步路径。墙钟减主线程 CPU 时间仍混合 GPU 等待、I/O 和被调度出去, NVML 也不是 GPU 执行计时; 这些观测用于缩小解释范围, 不预先判定根因。只有出现真实合法恢复事件与可重复动作差异后, 才进入第 5 节的单状态信号检验。

参考文献

- [1] Haiying Shen and Tanmoy Sen. Mitigating KV Cache Competition to Enhance User Experience in LLM Inference. arXiv:2503.13773v1, 2025. <https://arxiv.org/html/2503.13773v1>.
- [2] Rajveer Bachkaniwala et al. Stream2LLM: Overlap Context Streaming and Prefill for Reduced Time-to-First-Token. arXiv:2604.16395v1, 2026. <https://arxiv.org/html/2604.16395v1>. Official artifact: https://github.com/rajveerb/stream2llm/tree/mlsys_artifact.
- [3] Characterizing the Behavior and Impact of KV Caching on Transformer Inferences under Concurrency. Author-hosted full text, 2025. <https://jcernuda.com/publications/kv-caching>.
- [4] Shuowei Jin et al. Compute Or Load KV Cache? Why not Both? arXiv:2410.03065v2, 2025. <https://arxiv.org/html/2410.03065v2>.
- [5] Yuhan Liu et al. CacheGen: KV Cache Compression and Streaming for Fast Language Model Serving. SIGCOMM 2024; inspected arXiv:2310.07240v4. <https://arxiv.org/html/2310.07240v4>.
- [6] Zhang Cao et al. LUMEN: Coordinated Failure Recovery for Distributed LLM Serving. arXiv:2606.17787v1, 2026. <https://arxiv.org/html/2606.17787v1>.
- [7] Xingyu Xiang et al. ShadowServe: Interference-Free KV Cache Fetching for Distributed Prefix Caching. arXiv:2509.16857v1, 2025. <https://arxiv.org/html/2509.16857v1>.
- [8] OpenAI. Grade School Math, public test data. Selected test indices 128–191; file hashes preserved in the experiment freeze receipt. https://github.com/openai/grade-school-math/blob/master/grade_school_math/data/test.jsonl.
- [9] LongBench. Public `multifieldqa_en` data; revision `5e628be450b7e67fb7ae6e201bd6d8f7056f7672`. Selected 17 source rows and hashes preserved in the experiment freeze receipt. <https://huggingface.co/datasets/zai-org/LongBench>.